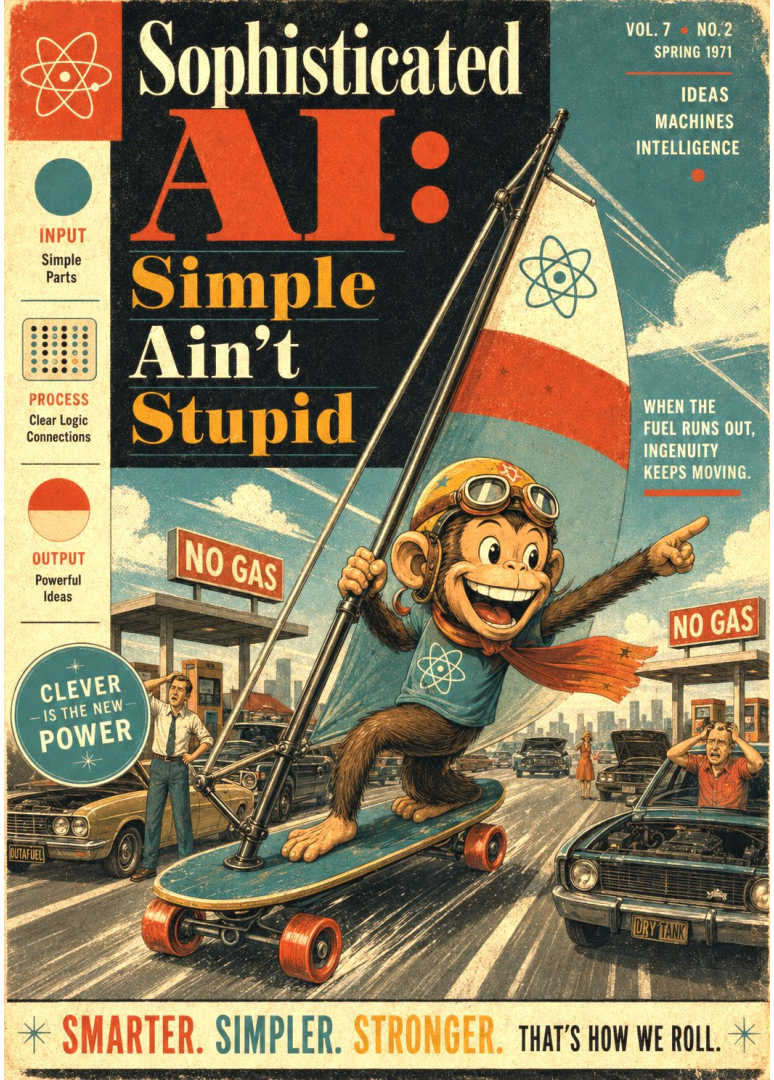
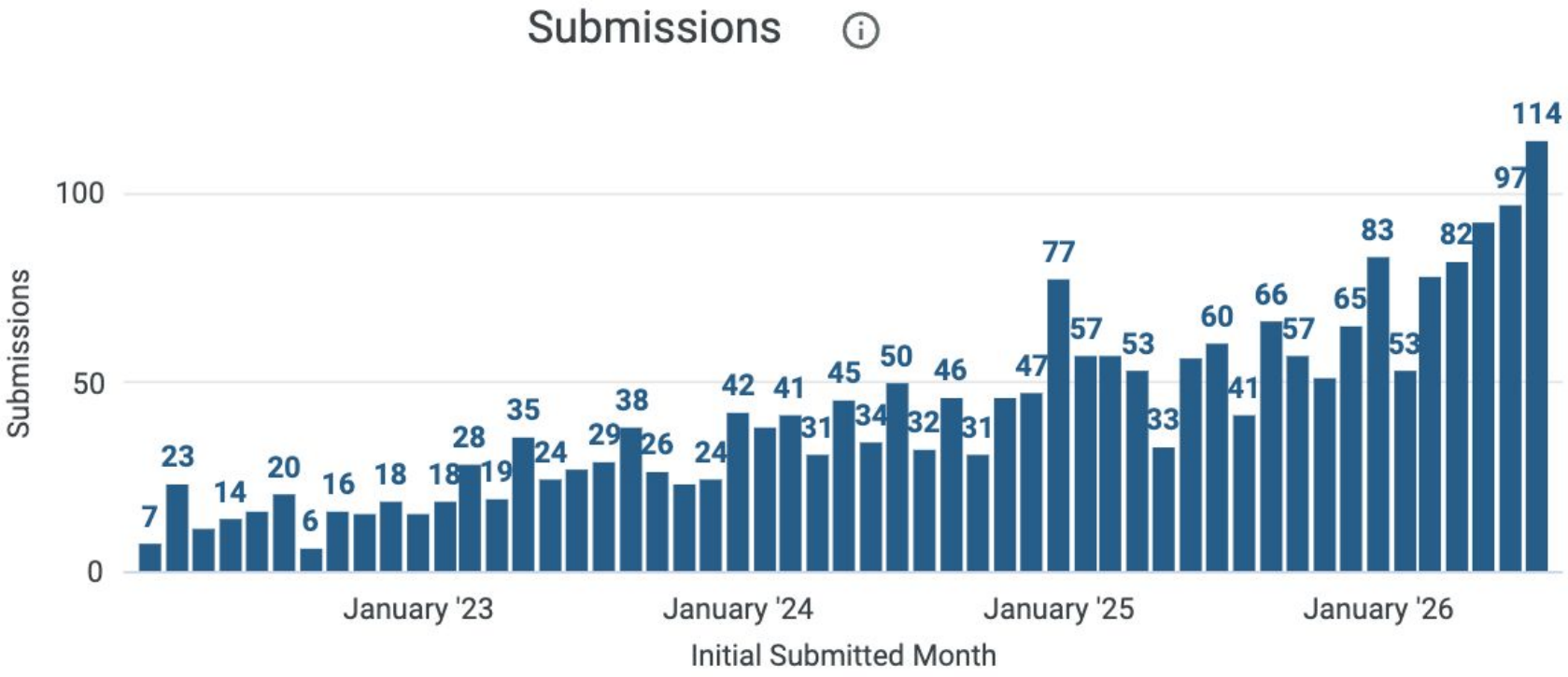

Sophisticated AI: Simple Ain't Stupid
VOL. 7 • NO. 2
SPRING 1971
IDEAS
MACHINES
INTELLIGENCE
INPUT
Simple Parts
PROCESS
Clear Logic Connections
OUTPUT
Powerful Ideas
WHEN THE FUEL RUNS OUT, INGENUITY KEEPS MOVING.
NO GAS
NO GAS
CLEVER IS THE NEW POWER
OUTAFUEL
DRY TANK
SMARTER. SIMPLER. STRONGER. THAT'S HOW WE ROLL.

Automated SE Journal



Tiny tool kit, reusable skills ⇒

Received: Added at production | Revised: Added at production | Accepted: Added at production

DOI: xxx/xxxx

RESEARCH PAPER

Can AI be Easy? Lessons Learned from the EZR.py Toolkit

Tim Menzies[1] | **Srinath Srinivasan**[1]

[1]Department of Computer Science, North Carolina State University, North Carolina, USA

[2]Department of Computer Science, North Carolina State University, North Carolina, USA

Correspondence
Tim Menzies,
Email: timm@ieee.org

Abstract

Much recent press claims that developers no longer need to read code. We disagree, at least within the domain of tabular software-engineering (SE) optimization tasks: rows of x and y values where the y values are expensive to obtain.

As evidence we present 400 lines of EZR.py, a Python toolkit (no heavy dependencies) that implements Naive Bayes, k-means clustering, classification and regression trees, simulated annealing, local search, active learning, and complementary-Bayes text-mining relevance filtering for tabular SE data. EZR was built by repeatedly reading and refactoring AI tools to simplify and unify them. The result demonstrates that many seemingly different learning algorithms are nearly the same once stripped back to their core: classical algorithms collapse to a few lines each, and a state-of-the-art active learner fits in roughly 80 lines.

Tested on the 120+ tabular SE optimization tasks in the MOOT repository, these tiny tools perform as well as or better than state-of-the-art explanation tools (SHAP, LIME), the SMAC3 optimizer, and SVM-based text-mining filters (FASTREAD), while running 500× faster than SMAC3, using orders of magnitude less labelled data, and building trees from fewer than ten variables even when thousands are available.

We conclude that, within the scope of tabular SE optimization, reading and refactoring code is a useful method of generating insight, and small unified toolkits can rival large libraries.

EZR is available under an open-source license. Install via `pip install ezr`; example data at github.com/timm/moot.

KEYWORDS

Active learning; explanation; multi-objective optimization; Complement Naïve Bayes; software analytics; minimalism; reproducibility; literate programming

04UV1 [cs.SE] 28 May 2026

https://arxiv.org/pdf/2606.03640

task	skill	ch	new LOC
Put 400 cars in one table; measure any gap	remember (representation)	3	120
Guess the fuel use before we see the sticker	guess (prediction)	4	35
Check our tricks on a hundred other lots	check (benchmarking)	5	0
Say when "better" is real, and when it is noise	certify (certification)	6	32
Decide on the spot as cars arrive one at a time	flow (streaming)	7	20
Say why the bad cars are slow, or thirsty	blame (diagnosis)	8	13
Justify any verdict to a skeptical buyer	justify (explanation)	9	12
Find the best car, test-driving very few	choose (optimization)	10	11
Chart a path from this car to a better one	route (planning)	11	5
Find the cheapest change that fixes this car	fix (repair)	12	11
Sketch a better car from halves of two others	blend (synthesis)	13	9
Spot a strange car: a typo, or a scam	spot (anomaly detection)	14	8
400 cars, one afternoon, what to inspect first?	ration (triage)	15	8
Trade fast against light against cheap	haggle (trade-off)	16	6
Notice when the lot quietly changes under us	watch (drift detection)	17	5
Shrink 400 cars to the dozen that summarize	shrink (compression)	18	3
Keep learning for years as the market moves	persist (lifelong learning)	19	7
Rehearse every skill on one knowable project	rehearse (simulation)	20	20
Answer the boss's morning questions with data	brief (analytics)	21	8
Race our skills against the field's best	race (baselining)	22	0
Teach the next intern to run the lot	teach (onboarding)	23	0

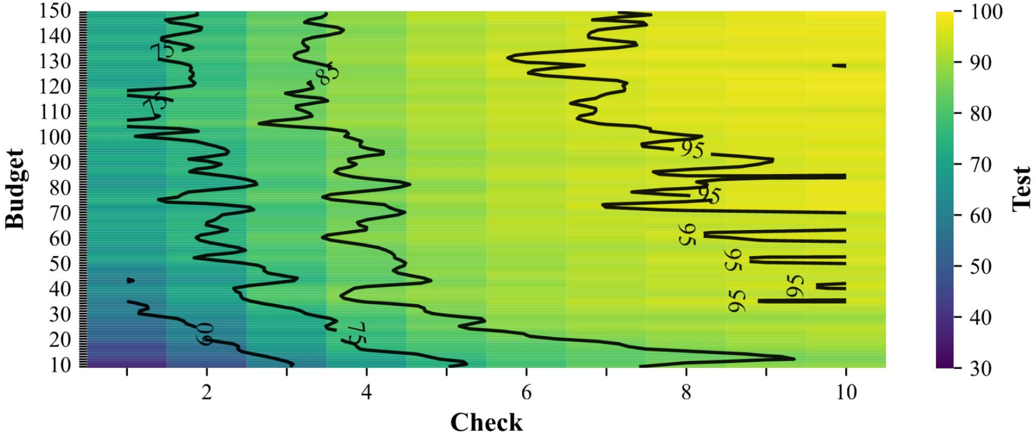
https://arxiv.org/pdf/2511.16882

Effective at optimization?

- Invariant: labelled data = sqrt(N) best, N-sqrt(N) rest
- **Budget** timesRepeat:
  - Label anything closer to best than rest
  - Update best rest
- Build binary tree on labels
- Use tree to sort holdout
- **Check** some items
- Return best of check

TABLE 1 The 124 multi-objective tasks in the MOOT repository[5] used in this paper. Tasks are expressed as tabular daa. Each row is a task category. #y is the number of goal columns; #x is the range of independent attribute. Data comes from recent papers from the SE literature (ICSE, FSE, TSE, IST, EMSE, TOSEM, and ASE[6,7,8,9,10,11,12,13,14,15,16]).

#	Category	Focus	File name(s)	Optimization challenge	#y	#x	Rows	Refs
Software systems, configuration, and tuning								
24	Soft. config	PLE	SS-[A-X]	Minimize footprint/memory vs. maximize throughput	2–3	3–88	197–86k	6
12	PromiseTune	Perf.	7z, LLVM, BDBC	Minimize execution time and energy consumption	1	6–35	864–166k	7
1	Cloud compute	Tuning	Apache_AllMeas	Balance server response vs. CPU/RAM load	1	9	192	8
1	Cloud compute	Tuning	SQL_AllMeas	Minimize latency/IO vs. maximize throughput	1	39	4,654	9
1	Cloud compute	Tuning	X264_AllMeas	Optimize encoding parameters for PSNR/SSIM	1	16	1,153	6
2	Testing	Testing	test120, test600	Maximize coverage while minimizing execution time	1	9	5,161	
Project management and process modeling								
35	Proj. health	Health	Health-Closed	Optimize PR rates and minimize developer churn	2–3	5	10,001	10
3	Scrum	Agile	Scrum[1k–100k]	Maximize velocity within sprint constraints	3	124	1k–100k	11
8	Feature mod.	Config	FFM, FM-*	Optimize clause/constraint ratio in large spaces	3	128–1k	10,001	6
1	nasa93dem	Cost	nasa93dem	Minimize effort (person-months) vs. quality	3	26	93	10
1	COC1000	Risk	coc1000	Minimize risks vs. analyst expertise	5	20	1,001	12
4	POM3	Process	pom3[a-d]	Balance project idle rates vs. completion costs	3	9	501–20k	11
4	XOMO	Defects	xomo_[flt,grd]	Minimize defect density vs. total project effort	4	27	10,001	12
Interdisciplinary and behavioral benchmarks								
4	Behavioral	Behavior	all_players, etc.	Maximize user retention and predict cohort churn	1–3	26–55	82–17k	13
4	Financial	Finance	BankChurners	Minimize credit risk vs. pricing models	2–5	19–77	1k–20k	14
3	Health	Health	COVID19, Life	Maximize accuracy/life vs. readmission likelihood	1–3	20–64	2k–25k	15
2	RL	Control	A2C_[Acr,Crt]	Maximize reward signals vs. steps to convergence	3–4	9–11	224–318	
5	Sales	Market	Marketing, socks	Maximize ROI vs. minimize forecasting error	1–8	20–31	247–2k	16
3	Misc.	General	auto93, Car	Optimize multivariate trade-offs (e.g. MPG vs. HP)	2–5	5–38	205–1k	6
124	**total**							



Faster that other optimizers

As good, or better, optimizations

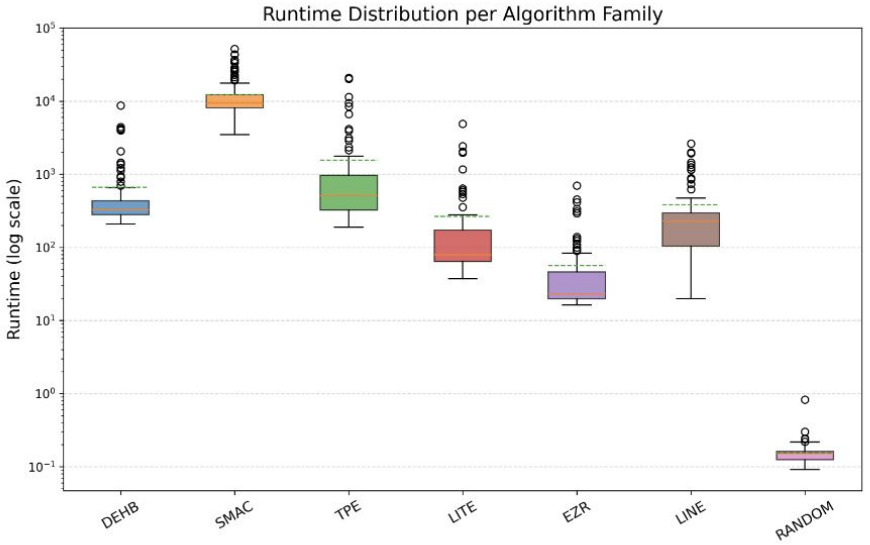


Fig. 5. Runtime comparison (log scale, milliseconds) of different optimizers across datasets.

https://arxiv.org/pdf/2509.08667

Effective at explanation?

RLF	SHAP	EZR	anova	all	data	x*	x	y	budget	rows
93	93	93	93	93	SS-A	3	3	2	150	1344
84	84	84	84	84	SS-B	2	3	2	80	207
84	84	84	84	84	SS-C	3	3	2	150	1513
77	77	77	77	77	SS-D	3	3	2	75	197
97	97	97	97	97	SS-E	3	3	2	150	757
98	98	98	97	98	SS-F	2	3	2	75	197
93	91	91	91	93	SS-G	2	3	2	75	197
80	99	99	99	99	wc-1	2	3	1	75	197
94	99	99	99	99	wc-2	2	3	1	75	197
94	99	99	99	99	wc-3	2	3	1	75	197
100	100	100	100	100	SS-H	2	4	2	100	260
99	99	99	99	99	SS-I	5	5	2	150	1081
80	80	80	79	79	auto93	3	5	3	150	399
66	66	66	66	66	HCI-hard	5	5	3	150	10001
100	100	100	100	100	HCI-easy	4	5	3	150	10001

White cells show the best performance; blue and yellow cells mark EZR answers that achieve at least 90% and 75% of the best; 40

https://dl.acm.org/doi/pdf/10.1145/3808192

Q: why so good?
A: BINGO effect

The real world is a special case.

Hides in the corners,
Behind the couch cushions

Generate bins using random projections.
N1=#bins with data

Generate bins2 with finer-grain
Projections. N2= #bins with data

Bingo effect N1 ≈ N2

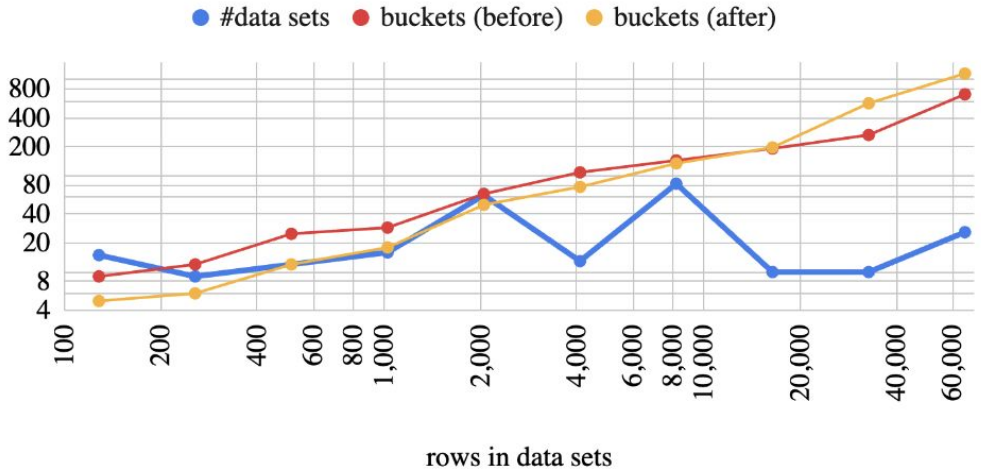
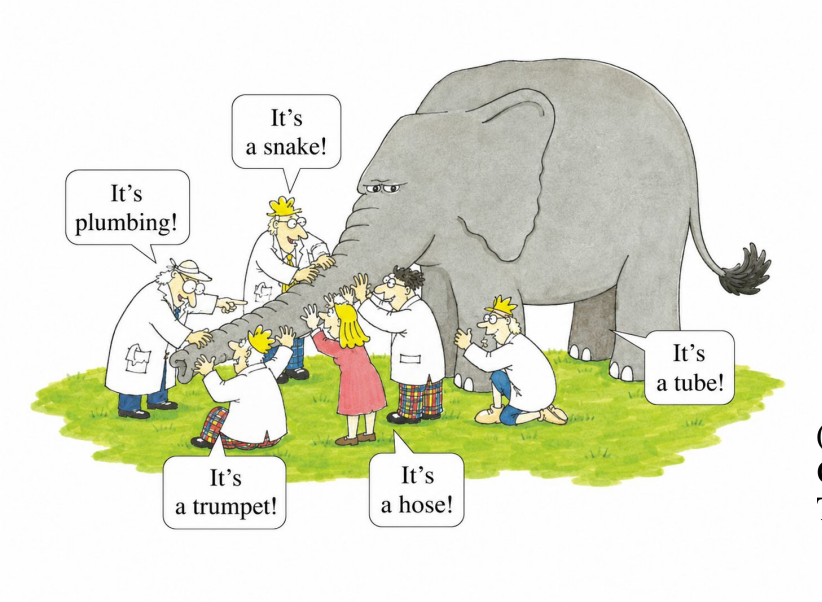
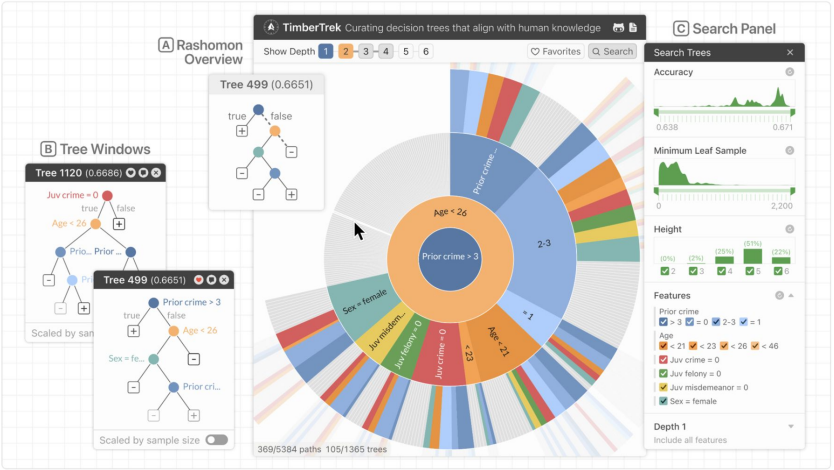


Fig. 7. Algorithm 1, applied to Table 1. *Buckets (after)* counts the filled buckets seen *after* increasing the data divisions. Y-axis shows mean results for data sets grouped by $\log_2$ of the number of rows. Blue lines indicate dataset sizes. Red lines show the bucket counts from randomly chosen bins/dimensions. Yellow

Rashomon sets (from Rudin et al).

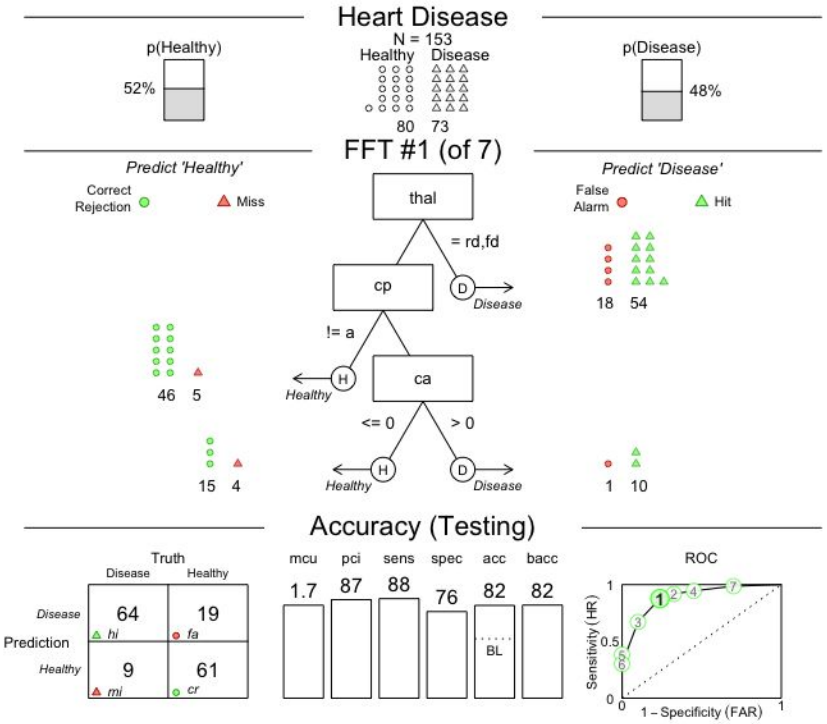
The (large) space of adequate models
That are close to the best one





(a) Rashomon set of the 1,365 best sparse decision trees for the COMPAS dataset, generated by TreeFARMS and displayed by TimberTrek (figure from Wang et al., 2022b).

FFTtrees (Phillipes et al).

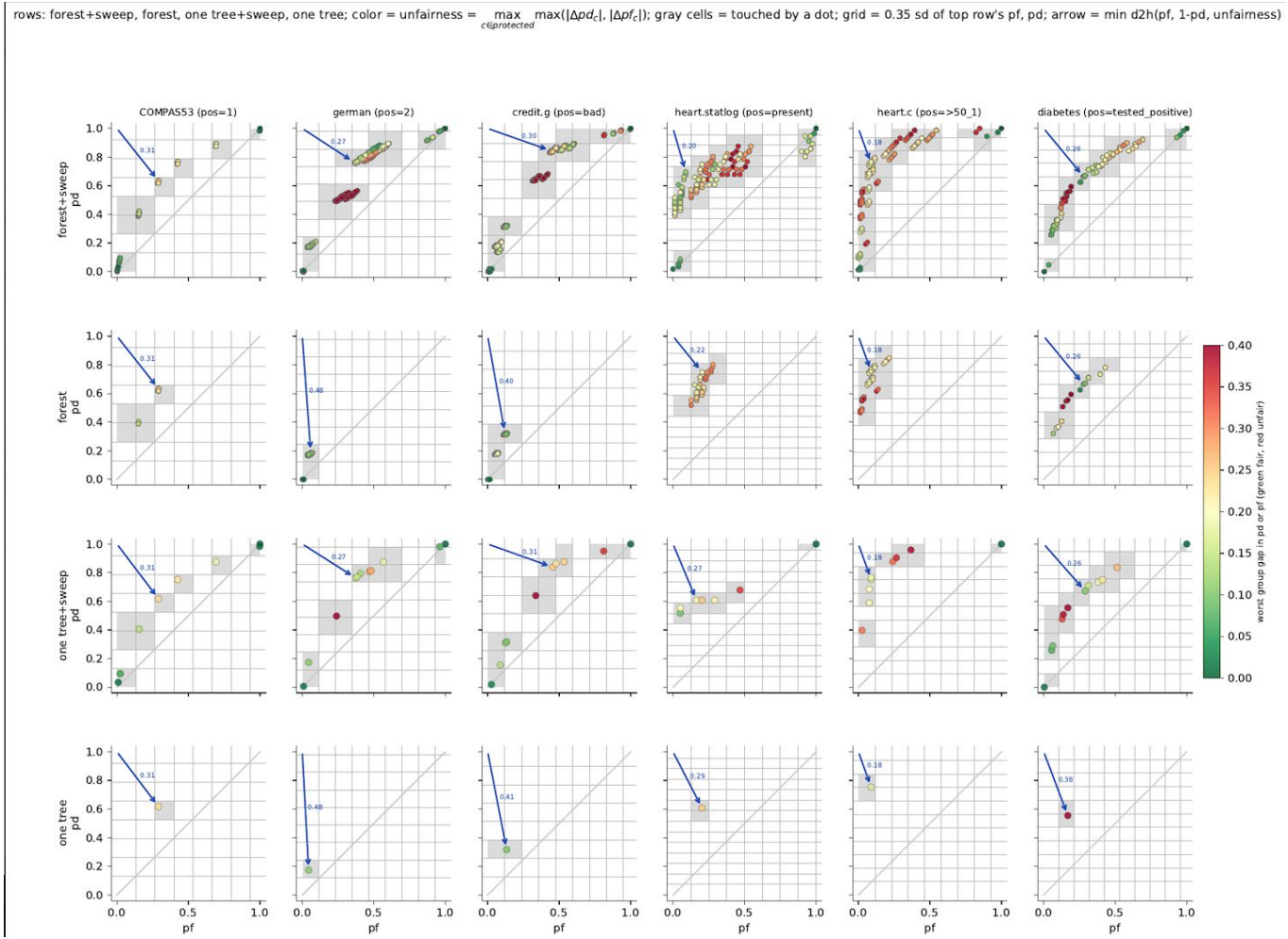


FFTstar trees (me)

FFT* trees

Generate as post processor to building binary tree

nearly isntant 3^5=243 trees



(un) Stability. The great secret

```
if TIME <= 4                    >   if CPLx <= 4                  >   if PCAP > 1                  >   if DATA > 3
|   if PCON > 3;               >   |   if PREC <= 5              >   |   if DOCU <= 2             >   |   if RUSE <= 3;
|   if PCON <= 3               >   |   |   if FLEx > 5;          >   |   |   if STOR > 5;         >   |   if RUSE > 3;
|   |   if ARCH <= 3           >   |   |   if FLEx <= 5          >   |   |   if STOR <= 5         >   if DATA <= 3
|   |   |   if LTEx > 2;       >   |   |   |   if PCON <= 2      >   |   |   |   if DOCU > 1      >   |   if PCON > 1
|   |   |   if LTEx <= 2;      >   |   |   |   |   if SITE > 2;  >   |   |   |   |   if PREC > 3;  >   |   |   if DATA > 2;
|   |   if ARCH > 3;           >   |   |   |   |   if SITE <= 2; >   |   |   |   |   if PREC <= 3; >   |   |   if DATA <= 2
if TIME > 4                     >   |   |   |   if PCON > 2;      >   |   |   |   if DOCU <= 1;    >   |   |   |   if DOCU > 4;
|   if SCED > 3;                >   |   if PREC > 5;              >   |   if DOCU > 2              >   |   |   |   if DOCU <= 4
|   if SCED <= 3                >   if CPLx > 4                   >   |   |   if SITE <= 3         >   |   |   |   |   if TIME > 4;
|   |   if PMAT > 3;           >   |   if PMAT <= 1;             >   |   |   |   if TEAM > 3;     >   |   |   |   |   if TIME <= 4;
|   |   if PMAT <= 3;          >   |   if PMAT > 1;              >   |   |   |   if TEAM <= 3;    >   |   if PCON <= 1;
                                >                                 >   |   |   if SITE > 3;         >
                                >                                 >   if PCAP <= 1;                >
```

Fig. 2: **Structural instability.** Example of models learned in this paper. Results are from 4 repeats varying only the random seed, extracted from [12], on Coc1000 dataset from MOOT (see Table III).

https://arxiv.org/pdf/2607.10420